

Tema 4:

Relojes y eventos

Cada ordenador tiene su propio **reloj local**. Pero como no están perfectamente sincronizados, surgen varios problemas:

- **Sesgo**: Diferencia entre dos relojes en un momento dado.
- **Deriva**: Uno de los relojes "va más rápido" o "más lento" que otro. Por ejemplo, puede atrasarse 1 segundo cada 108 días.

¿Qué hacemos entonces?

Utilizamos **sincronización**, que puede ser de dos tipos:

- **Externa**: Usamos una fuente como un reloj atómico (UTC, por satélite, radio, etc).
- **Interna**: Solo usamos los relojes de los ordenadores del sistema y los sincronizamos entre ellos.

MÉTODOS DE SINCRONIZACIÓN

1. Método de Cristian (1989)

Este método sirve para sincronizar un cliente con un servidor **que tenga la hora correcta (como UTC)**.

¿Cómo funciona?

1. El cliente le **pide la hora al servidor**.
2. El servidor responde con su hora TUTC.
3. El cliente mide cuánto tardó en llegar la respuesta (TVIAJE).
4. Como no sabemos exactamente cuánto tardó el mensaje en ir y volver, **se estima que tardó la mitad en cada dirección**.
5. El cliente **ajusta su reloj así**:

$$T_{\text{cliente}} = T_{\text{UTC}} + (TVIAJE / 2)$$



Precisión

- Si sabes el **tiempo mínimo real de ida**, puedes mejorar la estimación:

$$\text{Precisión} = \pm (TVIAJE / 2) - T_{\text{min}}$$

⚠ Inconvenientes

- Si el servidor UTC falla, el algoritmo deja de funcionar.
 - Cristian propuso usar varios servidores y quedarse con la **respuesta más rápida**, aunque eso puede ser arriesgado si el más rápido se equivoca.
-

2. 🧑 Método de Berkeley

A diferencia de Cristian, **no se usa un reloj externo**, sino que los ordenadores **se sincronizan entre ellos**.

¿Cómo funciona?

1. Se elige un **coordinador** (maestro).
2. El maestro **pregunta la hora a los demás (esclavos)**.
3. Calcula el tiempo de ida y vuelta de cada mensaje.
4. Hace una **media** de todos los relojes, **descartando** los que están demasiado desviados.
5. Envía a cada esclavo un **ajuste** (por cuánto deben adelantar o atrasar su reloj).

🔧 Características

- No se manda una nueva hora, sino un **factor de corrección**.
 - Si el coordinador falla, **se elige otro**. Por eso, es un método de **sincronización interna**.
-

3. 🌐 NTP (Network Time Protocol)

Este método se usa para **Internet**, a gran escala.

¿Cómo funciona?

- Usa **varios niveles (estratos)**. El estrato 1 tiene acceso directo al UTC.
- Los demás niveles se sincronizan con los superiores.
- En cada paso, se aplica el **algoritmo de Cristian**, pero con filtros estadísticos y redundancia.

💡 Ventajas

- **Tolera fallos** de conexión largos.
- Usa muchos servidores para mejorar precisión.
- **Filtra errores**, incluso ataques maliciosos.
- **Sincroniza frecuentemente** para evitar desviaciones.

TIEMPO LÓGICO (Lamport)

🧠 ¿Por qué usar tiempo lógico?

- En sistemas distribuidos **no se puede garantizar que todos los relojes estén sincronizados perfectamente.**
- Entonces **no podemos usar el tiempo real para saber quién hizo algo antes.**
- En lugar de eso, usamos una **regla de causalidad**: si una cosa **causa** otra, debe haber pasado antes.

🕒 Relojes lógicos de Lamport (1978)

Cada proceso tiene su **propio reloj lógico** L_i , que no mide tiempo real, sino el **orden de los eventos**.

📋 Reglas:

1. Cada vez que el proceso hace algo $\rightarrow L_i = L_i + 1$
2. Cuando envía un mensaje, manda L_i junto con el mensaje.
3. Cuando otro proceso recibe ese mensaje:

$$L_i = \max(L_i, \text{Trecibido}) + 1$$

Así, se asegura de que el evento de recepción **ocurra después** del envío.

✅ Ventaja:

- Nos permite decir: *“esto ocurrió antes que aquello”* sin usar relojes reales.

🎮 Ejemplo simple:

Supón que tienes dos procesos A y B:

A: Evento1 --- (msg:3) --> Evento2

B: recibe msg

- En A, antes de enviar el mensaje, el reloj vale 3.
- B recibe el mensaje. Su reloj valía 2, pero ahora hace:

$$B = \max(2, 3) + 1 = 4$$

Así B sabe que ese mensaje vino **de algo que ocurrió en el tiempo 3**, y ajusta su reloj para reflejarlo.

Relojes vectoriales (versión mejorada)

- En vez de un solo número L_i , cada proceso lleva un **vector** con **N posiciones** (una por cada proceso del sistema).
- Así puede saber exactamente **qué procesos han influido en cada evento**.

Ejemplo con 3 procesos:

- Un proceso tiene el vector $[2, 1, 0]$ → significa:
 - Yo (P1) he hecho 2 eventos,
 - Sé que P2 hizo 1,
 - Y que P3 no ha hecho nada que me haya afectado.

Ventaja:

- Nos permite detectar **conurrencia**: si dos eventos no se conocen entre sí, son concurrentes.

Desventaja:

- Requiere más **espacio de memoria** (guardar vectores en lugar de números).
- Los **mensajes son más grandes**.



EXCLUSIÓN MUTUA EN SISTEMAS DISTRIBUIDOS

🌱 ¿Qué es?

Es un mecanismo para **evitar que dos procesos accedan al mismo recurso al mismo tiempo**, como por ejemplo:

- Un archivo,
- Una base de datos,
- Una impresora...

Esto se gestiona mediante una **sección crítica**: una zona del programa donde se accede a ese recurso, y **solo un proceso puede estar dentro a la vez**.

✅ Requisitos básicos para exclusión mutua:

1. **Seguridad**: Solo **un proceso a la vez** puede estar en la sección crítica.
 2. **Pervivencia**: Si alguien quiere entrar, **algún día podrá hacerlo** (no se queda esperando para siempre).
 3. **Orden**: Si uno pide antes que otro, **entra primero**.
-

Ahora veremos **varios algoritmos** para lograr esto. Vamos uno a uno 🙌

1. 🌀 Algoritmo del Servidor Central

📌 ¿Cómo funciona?

- Hay un **servidor central** que guarda el "testigo" (permiso de acceso).
- Un proceso que quiere entrar en la sección crítica le **pide el testigo al servidor**.
- Si otro proceso ya lo tiene, el servidor **pone al nuevo en cola**.
- Cuando el proceso actual **sale**, devuelve el testigo, y el servidor se lo pasa al siguiente.

🚧 Evaluación

- **Fácil de implementar**.
 - Pero... si el servidor **se cae**, nadie más puede entrar.
 - También puede haber **cuello de botella** (el servidor recibe todas las peticiones).
-

2. Algoritmo del Anillo

¿Cómo funciona?

- Los procesos están conectados en un **anillo circular**.
- El testigo **viaja por el anillo**.
- Si un proceso **necesita entrar**, se **queda con el testigo** cuando le llega.
- Si no necesita entrar, **lo pasa al siguiente**.

Evaluación

- Evita servidor único (mejor distribución).
 - **El testigo circula siempre**, aunque nadie lo necesite → gasto de red.
 - Si un nodo falla, hay que **reorganizar el anillo**.
-

3. Algoritmo por Ordenación de eventos

¿Cómo funciona?

- Cada proceso **manda su petición con marca de tiempo**.
- Entra en la sección crítica **el que tiene la marca más baja**.
- Al salir, **avisa a los demás** para que entren por orden.

Evaluación

- **Muchos mensajes** (hay que avisar a todos).
 - Mucho más **preciso en el orden**, pero **más costoso en ancho de banda**.
-

4. Algoritmo de Maekawa

¿Cómo funciona?

- No hace falta que todos los procesos den permiso.
- Solo se necesita un **grupo de votos** (subconjunto).
- Si un proceso consigue mayoría, **entra** en la sección crítica.

Problema:

- Si hay empate de votos o muchos procesos quieren entrar a la vez, puede haber **bloqueo (deadlock)**.
- Hay soluciones, pero hacen el algoritmo **más complejo**.